Análise Empírica de Complexidade Algorítmica

Análise Assintótica

A análise assintótica é uma forma de estudar como uma função ou algoritmo se comporta quando o tamanho da entrada cresce muito. Em vez de focar no tempo exato de execução, ela observa a tendência de crescimento, ignorando detalhes como constantes e termos menores. Assim, conseguimos comparar algoritmos de maneira mais simples e prever qual será mais eficiente para grandes volumes de dados, usando notações como $O(n)$, $O(n^2)$ e $\Theta(n)$.

ref: https://www.complexica.com/narrow-ai-glossary/asymptotic-analysis

Notação O (Big O)

A notação Big O é uma forma matemática de descrever como o tempo de execução (ou uso de memória) de um algoritmo cresce conforme o tamanho da entrada aumenta. Ela representa o limite superior desse crescimento, ou seja, o pior cenário esperado.

Em termos simples, a Big O mostra a taxa de crescimento dominante de um algoritmo, ignorando constantes e termos menores. Por exemplo, um algoritmo com complexidade $O(n^2)$ indica que o número de operações cresce proporcionalmente ao quadrado do tamanho da entrada, ajudando a prever seu desempenho em grandes volumes de dados.

ref: https://www.freecodecamp.org/portuguese/news/o-que-e-a-notacao-big-o-complexidade-de-tempo-e-de-espaco/

Análise dos Algoritmos

Algoritmo A (for duplo)

Melhor caso:

Ocorre quando existe um elemento duplicado logo no início do vetor. A função encontra a repetição rapidamente e retorna imediatamente. Complexidade **O(1)**.

Pior caso:

Acontece quando não há elementos duplicados. O algoritmo compara todos os pares possíveis com dois loops aninhados. Complexidade **$O(n^2)$**.

Caso médio:

Com dados aleatórios, duplicatas podem existir, mas geralmente não aparecem no início. O algoritmo ainda realiza muitas comparações. Complexidade **$O(n^2)$**.

Algoritmo B (ordenar + verificar)

Melhor caso:

Mesmo que o vetor já esteja ordenado e tenha duplicatas no início, o algoritmo sempre executa a ordenação. O custo é dominado pelo `qsort`. Complexidade **O(n log n)**.

Pior caso:

Depende do comportamento do `qsort`, que pode degradar em algumas implementações. Após ordenar, o vetor é percorrido completamente. Complexidade **$O(n^2)$** no pior caso.

Caso médio:

O `qsort` geralmente executa em **O(n log n)**, e a verificação é linear. Complexidade **O(n log n)**.

Algoritmo C (hash)

Melhor caso:

Ocorre quando um duplicado aparece logo no início. A verificação detecta rapidamente a repetição. Complexidade **O(1)**.

Pior caso:

Quando não há duplicatas, todos os elementos são percorridos e inseridos na estrutura auxiliar. Complexidade **O(n)**.

Caso médio:

Para entradas aleatórias, o algoritmo percorre parte significativa do vetor até encontrar duplicatas ou terminar. O comportamento esperado é linear. Complexidade **O(n)**.

Análise de Resultados

Tabela de tempos de execução

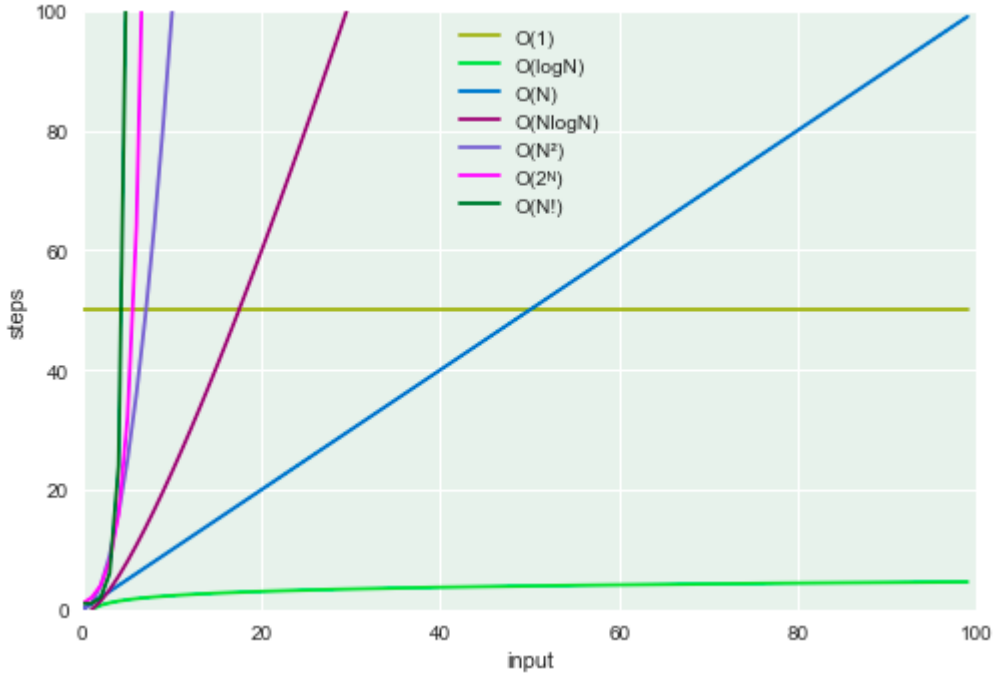
N	Algoritmo A (s)	Algoritmo B (s)	Algoritmo C (s)
5.000	0.05	0.001	0.006
20.000	0.45	0.008	0.006
50.000	1.19	0.01	0.007

Análise dos resultados

O Algoritmo A apresenta um crescimento rápido do tempo de execução conforme o tamanho da entrada aumenta. O salto de 0.05 para 1.19 indica um comportamento quadrático, coerente com a complexidade $O(n^2)$. Isso mostra que ele se torna inviável para grandes volumes de dados.

O Algoritmo B apresenta crescimento moderado. O tempo aumenta conforme N cresce, mas de forma bem mais controlada. Isso confirma o comportamento esperado de O(n log n), típico de algoritmos baseados em ordenação.

O Algoritmo C mantém tempos praticamente constantes entre os diferentes tamanhos de entrada. Isso indica um comportamento linear O(n), com excelente desempenho e pouca variação, sendo o mais eficiente na prática entre os três.



Conclusão

Os resultados confirmam a teoria da análise assintótica. O Algoritmo A, com complexidade quadrática, é o menos eficiente e escala mal com o aumento dos dados. O Algoritmo B apresenta desempenho intermediário, sendo mais eficiente que o A, mas ainda limitado pelo custo da ordenação. Já o Algoritmo C é o mais eficiente, com crescimento linear e tempos praticamente constantes, sendo a melhor escolha para grandes volumes de dados.